

Learning Value of Information using Reinforcement Learning

Lokesh Boominathan

1 Motivation

At the crux of reinforcement learning problems is the trade-off between exploration and exploitation. Learning about the environment would require information-seeking actions that might be different from actions that maximize the reward given current knowledge of the world. Such information-seeking actions may involve interactions with the oracle (such as humans), which can provide either clues for task-relevant variables, or even give guidance on the policy to be learned. However, choosing information-seeking actions can also come with its own cost (such as the cost of hiring mechanical turkers). The goal of this project is to understand the value of performing such information-seeking actions, so as to better understand the exploration-exploitation trade-off. To this end, we focus on the toy problem described in Howard's seminal paper on "Information value theory" [1]. For the toy problem, Howard showed that Bayesian decision theory can be used to estimate the value of information-seeking actions, given prior distributions encoding partial knowledge. We explore how the problem can be posed as a Reinforcement Learning problem, and if the solution converges to the derived closed form expressions.

2 Problem

In [1], Howard presented a worked example in which a company makes a price bid (denoted as b) for a contract auction. The company gets the contract only if its bid is lower than the lowest bid offered by its competitors (denoted as l). And in the case the company gets the contract, the profit it makes is the difference between its bid and the cost-incurred (denoted as p) in performing the contract operation. The company only has prior distributions over its competitor's lowest bid, and the cost to be incurred, and does not know the exact values. However, before arriving at company's bid, the company has the option to hire a spy for a certain cost $Cost_{spy}$, so as to know the exact value of the lowest bid of its competitors. The question of interest is until what value of $Cost_{spy}$ is it reasonable to hire a spy, and what should be the company's bid be? The highest tolerable $Cost_{spy}$ would be then the value of information.

3 Approach 1: Modeling as Markov decision process

Our first approach models the above problem as a Markov decision process (MDP) problem, and uses a policy-gradient based method to solve the MDP. We use a gradient based method as the parameter space, described below, is continuous. The fully observable state in the MDP case is the spy cost. The possible actions are to either directly come up with a bid without spying, or to spy and then come up with the bid.

3.1 Case 1: Fixed spy cost, and optimal action on spying

We first consider the simpler case where the spy cost is fixed for all the episodes, and we enforce the company to choose the theoretically optimal bid in the case it chooses to spy. The theoretical optimal bid would be a value slightly lower than the competitors' minimum bid, if the average cost to be incurred is lower than the minimum competitors' bid. However, if average cost to be incurred is higher than the competitors' minimum bid, than the company either chooses not to bid, or equivalently bids a very high value so as to not get the contract. Thereby avoiding the high chances of incurring loss.

Problem parametrization: The action choice of whether to spy or not is parametrized by a learn-able Bernoulli mean parameter. The outcome of a Bernoulli random draw with this mean would then determine whether to spy or not. Likewise, in the case the company chooses not to spy, the bid is based on a learn-able Gaussian mean parameter. The bid value will be a sample from the Gaussian distribution described by this mean, and a fixed variance. We chose to make theses actions stochastic as it encourages for controlled exploration (in terms of variance). The setup is schematically shown in Fig 1.

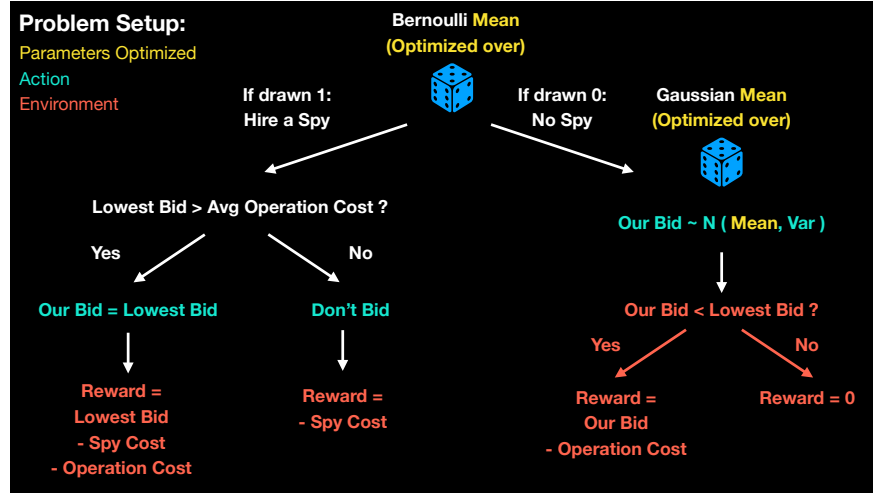


Figure 1: *Parametrization of the MDP problem, when optimal bid in the case of spying is enforced. The Bernoulli and Gaussian means are the parameters optimized for. They represent the probability of spying, and the probability of choosing a particular bid value in the case of not spying respectively.*

Learning: Since the parameters to be optimized for are from continuous domain, we cannot use algorithms such as value iteration which are useful only when the actions choices are discrete. Hence, we resort to a policy-gradient based method where the derivative of

the total future reward (denoted as J) is taken with respect to the parameters (denoted as θ). The parameters are then moved in the direction of gradient so as to find the optimal parameters (Eqs1-2).

$$J(\theta) = E \left[\sum_{t=0}^{T-1} r_{t+1} \right] \quad (1)$$

$$\theta \leftarrow \theta + \frac{\partial}{\partial \theta} J(\theta) \quad (2)$$

The gradient is computed using the REINFORCE algorithm [2], with the derivation described in [2] summarized as below.

$$\begin{aligned} J(\theta) &= E \left[\sum_{t=0}^{T-1} r_{t+1} \mid \pi_{\theta} \right] \\ &= \sum_{t=0}^{T-1} P(s_t, a_t \mid \tau) r_{t+1}, \end{aligned}$$

where π_{θ} denotes the parametrized policy and $P(s_t, a_t \mid \tau)$ denotes the joint probability of state s_t and action a_t , given the trajectory τ . On computing the derivative, we have

$$\nabla_{\theta} J(\theta) = \sum_{t=0}^{T-1} \nabla_{\theta} P(s_t, a_t \mid \tau) r_{t+1}.$$

However, above can be difficult to compute since Monte Carlo simulation cannot be directly applied. Hence, we rewrite the derivative in terms of $P(s_t, a_t \mid \tau)$ as below.

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \sum_{t=0}^{T-1} \nabla_{\theta} P(s_t, a_t \mid \tau) r_{t+1} \\ &= \sum_{t=0}^{T-1} P(s_t, a_t \mid \tau) \frac{\nabla_{\theta} P(s_t, a_t \mid \tau)}{P(s_t, a_t \mid \tau)} r_{t+1} \\ &= \sum_{t=0}^{T-1} P(s_t, a_t \mid \tau) \nabla_{\theta} \log P(s_t, a_t \mid \tau) r_{t+1} \\ &= E \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log P(s_t, a_t \mid \tau) r_{t+1} \right] \end{aligned}$$

. Therefore we can compute the derivative by taking expectation over the different samples drawn. Note that the entity $P(s_t, a_t \mid \tau)$ can also be easily computed since we parameterized the actions as those drawn from Bernoulli and Gaussian distributions. Once we have the derivative, we then perform gradient ascent to finally arrive at the optimal parameter values.

Result: As shown in Fig 2, the optimal choice of when to spy/not spy matches with that obtained using Howard's solution. The average profit obtained is also the same, implying the agent's bid when it chose not to spy is similar to Howard's optimal bid. We see that a spy cost of 0.3 is where the optimal choice transitions from spying to not spying. Therefore, 0.3 is attributed as the value of information, as this is the highest amount worth paying so as to obtain the information of minimum bid.

Spy Cost	0	0.1	0.2	0.3	0.4	0.5	0.6
Our Freq Action	Spy	Spy	Spy	Spy	No Spy	No Spy	No Spy
Howard Action	Spy	Spy	Spy	Either	No Spy	No Spy	No Spy
Our Avg Profit $\pm$ SEM*	0.564 ± 0.005	0.46 ± 0.005	0.37 ± 0.005	0.26 ± 0.005	0.26 ± 0.004	0.26 ± 0.004	0.26 ± 0.004
Howard Avg Profit	0.58	0.48	0.38	0.28	0.28	0.28	0.28

Figure 2: For a fixed spy cost (each column), The agent successfully learned the optimal action of when to spy, and how much to bid if it does not choose to spy. As shown in the table above, the agent’s actions and average profit matches that shown by Howard. SEM stands for Standard Error of Mean.

3.2 Case 2: Varying spy cost, and optimal action on spying

In a similar way we solve for the setting where the spy cost can change for every episode, while still enforcing the company to choose the theoretically optimal bid in the case it chooses to spy.

Result: In Fig 3, we plot the probability of bidding right away (not spying) as a function of spy cost. Since 0.3 was earlier calculated to be the theoretical value of information, we should ideally observe a step function with a jump from probability 0 to 1 at a spy cost of 0.3. In Fig 3 we see that the learned agent deviates slightly from the ideal case, having more of a sigmoid form than a step function, with the transition starting at a spy cost of around 0.3.

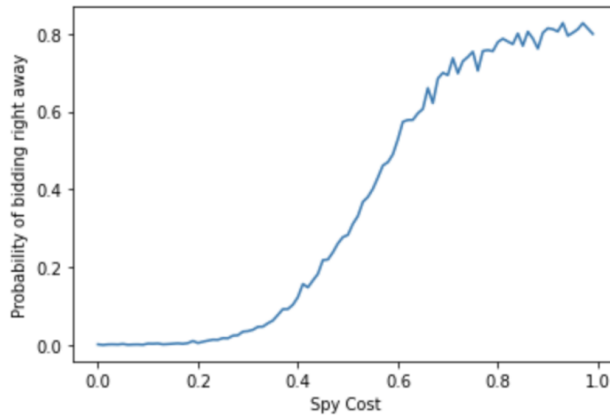


Figure 3: Probability of bidding right away (not spying) plotted as a function of spy cost. The transition of probability from 0 to higher values start at a spy cost of around 0.3, the theoretical value of information.

4 Approach 2: Modeling as a Partially observable Markov decision process

In the second approach, we formulate the problem as a Partially observable Markov decision process (POMDP) instead. The reason being that the competitors' lowest bid is observable only when the agent's action is to spy. The components of the state vector in this case are the competitors' minimum bid, and the cost to be incurred. If the company chooses the action of 'spying', then it gets the value of competitors' minimum bid as the observation. If the action is to bid, then the state does not change. If the action is to bid, then the state changes to another set of random values drawn from the prior distribution, indicating the next episode. The reward model is as before. The possible actions are either to spy, or to choose a particular quantized value as the company bid. We quantize the possible values of company bid so as to reduce the action space from continuous space to finite number of discrete points. Mathematically, we can describe the POMDP using the tuple (S, A, Ω, T, O, R) as below.

- State set $S : [s_1 \ s_2]$, where $s_1 \in [0, l_{max}]$, $s_2 \in [0, p_{max}]$
- Action set $A : \{spy, bid = \frac{l_{max}}{N}, bid = 2\frac{l_{max}}{N}, \dots, bid = l_{max}\}$, where company bid is quantized into N possible values.

- Observation set $\Omega : [0, l_{max}]$

- Transition $T(s' | s, a)$:

$$a = spy \implies s' = s$$

$$a = bid \ b \implies s' = [s'_1 \sim U(0, l_{max}), s'_2 \sim U(0, p_{max})]$$

Here U denotes uniform distribution.

- Observation $O(o | s, a)$:

$$a = spy \implies o = s_1$$

$$a = bid \ b \implies o = null$$

- Reward $R(s, a)$:

$$a = spy \implies R = -Cost_{spy}, \text{ where } Cost_{spy} > 0$$

$$a = bid \ b \implies R = \begin{cases} 0, & \text{if } b > s_1 \\ b - s_2, & \text{if } b \leq s_1 \end{cases}$$

4.1 POMDP expressed as MDP in belief space

Here we derive how the above POMDP can be expressed as an MDP in the belief space. From the above formulation, we see that future state is a function of current state and action, observation is a function of state and action, and the belief over state (expressed using sufficient statistic ϕ) is a function of previous belief, action, and current observation. This can be described using the graphical model shown in trellis Fig 4.

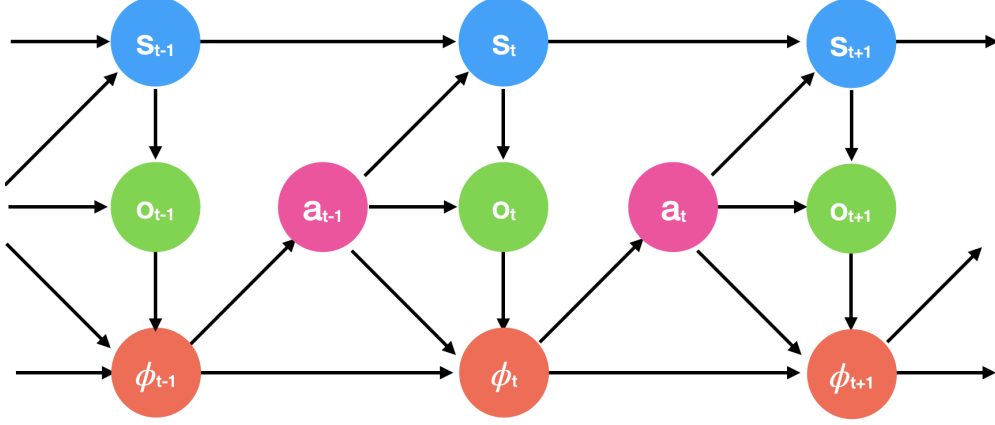


Figure 4: *Howard's problem formulated as a Partially observable Markov decision process (POMDP) using the above graphical model. Notations s , o , a , and ϕ , are used to represent the states, actions, observations, and beliefs respectively.*

Let $B(s_t|\phi_t)$ represent the probability of state s_t given the sufficient statistic ϕ_t . Knowing ϕ_t is equivalent to knowing the trajectory of observations and action until that time. Hence we can write

$$\begin{aligned}
 B(s_t|\phi_t) &= B(s_t|o_t, a_{t-1}, o_{1:t-1}, a_{1:t-2}) \\
 &= B(s_t|o_t, a_{t-1}, \phi_{t-1}) \\
 &= \frac{Pr(o_t|s_t, a_{t-1}, \phi_{t-1}) Pr(s_t|a_{t-1}, \phi_{t-1})}{Pr(o_t|a_{t-1}, \phi_{t-1})}
 \end{aligned}$$

Since $Pr(o_t|a_{t-1}, \phi_{t-1})$ is not a function of s_t , we can treat it as a constant Z . Also, from graphical model we have $Pr(o_t|s_t, a_{t-1}, \phi_{t-1}) = Pr(o_t|s_t, a_{t-1})$, $Pr(s_t|s_{t-1}, a_{t-1}, \phi_{t-1}) = Pr(s_t|s_{t-1}, a_{t-1})$, $Pr(s_{t-1}|a_{t-1}, \phi_{t-1}) = Pr(s_{t-1}|\phi_{t-1})$. Simplifying further using above.

$$\begin{aligned}
 B(s_t|\phi_t) &= \frac{1}{Z} Pr(o_t|s_t, a_{t-1}) Pr(s_t|a_{t-1}, \phi_{t-1}) \\
 &= \frac{1}{Z} Pr(o_t|s_t, a_{t-1}) \sum_{s_{t-1}} Pr(s_t, s_{t-1}|a_{t-1}, \phi_{t-1}) \\
 &= \frac{1}{Z} Pr(o_t|s_t, a_{t-1}) \sum_{s_{t-1}} Pr(s_t|s_{t-1}, a_{t-1}, \phi_{t-1}) Pr(s_{t-1}|a_{t-1}, \phi_{t-1}) \\
 &= \frac{1}{Z} Pr(o_t|s_t, a_{t-1}) \sum_{s_{t-1}} Pr(s_t|s_{t-1}, a_{t-1}) Pr(s_{t-1}|\phi_{t-1}) \\
 &= \frac{1}{Z} O(o_t|s_t, a_{t-1}) \sum_{s_{t-1}} T(s_t|s_{t-1}, a_{t-1}) B(s_{t-1}|\phi_{t-1}) \tag{3}
 \end{aligned}$$

The observation probability in belief space is derived as below.

$$\begin{aligned}
\bar{O}(o_{t+1}|\phi_t, a_t) &= \sum_{s_{t+1}} Pr(o_{t+1}, s_{t+1}|\phi_t, a_t) \\
&= \sum_{s_{t+1}} Pr(o_{t+1}|s_{t+1}, \phi_t, a_t) Pr(s_{t+1}|\phi_t, a_t) \\
&= \sum_{s_{t+1}} Pr(o_{t+1}|s_{t+1}, a_t) \sum_{s_t} Pr(s_{t+1}, s_t|\phi_t, a_t) \\
&= \sum_{s_{t+1}} Pr(o_{t+1}|s_{t+1}, a_t) \sum_{s_t} Pr(s_{t+1}|s_t, \phi_t, a_t) Pr(s_t|\phi_t, a_t) \\
&= \sum_{s_{t+1}} Pr(o_{t+1}|s_{t+1}, a_t) \sum_{s_t} Pr(s_{t+1}|s_t, a_t) Pr(s_t|\phi_t) \\
&= \sum_{s_{t+1}} O(o_{t+1}|s_{t+1}, a_t) \sum_{s_t} T(s_{t+1}|s_t, a_t) B(s_t|\phi_t)
\end{aligned} \tag{4}$$

The transition probability in belief space is derived as below.

$$\begin{aligned}
\bar{T}(\phi_{t+1}|\phi_t, a_t) &= \sum_{o_{t+1}} Pr(\phi_{t+1}, o_{t+1}|\phi_t, a_t) \\
&= \sum_{o_{t+1}} Pr(\phi_{t+1}|o_{t+1}, \phi_t, a_t) Pr(o_{t+1}|\phi_t, a_t) \\
&= \sum_{o_{t+1}} Pr(\phi_{t+1}|o_{t+1}, \phi_t, a_t) \bar{O}(o_{t+1}|\phi_t, a_t)
\end{aligned} \tag{5}$$

Note that $Pr(\phi_{t+1}|o_{t+1}, \phi_t, a_t)$ in the above expression can be written as $Pr(\phi_{t+1}|o_{t+1}, \phi_t, a_t) = \delta(\phi_{t+1} - HMM(o_{t+1}, \phi_t, a_t))$. Here $HMM(o_{t+1}, \phi_t, a_t)$ is the hidden markov model given by right hand side of the equality in Eq 3. In our problem, this further simplifies because if the action is to spy, then ϕ_{t+1} is such that $Pr(s_1 = o_{t+1}) = 1$ and $Pr(s_2)$ is $U(0, p_{max})$. And if the action was to bid, because it's a new episode at the next time step, we have $Pr(s_1)$ is $U(0, l_{max})$, and $Pr(s_2)$ is $U(0, p_{max})$.

And finally the reward function in belief space can be written as

$$\begin{aligned}
\bar{R}(\phi_t, a_t) &= \sum_{s_t} R(s_t, a_t) Pr(s_t|\phi_t) \\
&= \sum_{s_t} R(s_t, a_t) B(s_t|\phi_t)
\end{aligned} \tag{6}$$

Note that unlike the state, the belief is fully observable. Moreover, in Eqs 3-6, we derived the transition probability $\bar{T}$ and the reward function $\bar{R}$ in belief space. Therefore, we converted the original POMDP problem in state space as an MDP problem in the belief space. However, solving this MDP can be still be quite challenging. This is because, although we approximated the possible values of competitors' lowest bid to N bins, now each bin has a probability value ranging between 0 to 1. To keep it discrete, the probability values also need to be quantized to, say M number of bins. For reasonable approximation M and N should not be too small, making the belief update quite complex.

4.2 Using POMDP solver package directly

I also tried applying a very recently released python POMDP solver package *pomdp-py* [3] to solve Howard’s problem. Although it seems to be that I have set up the problem in a way similar to the example POMDP problem shown in their website, there are issues in running the solver. I have attached my code for Howard’s problem in Appendix A.3.

5 Conclusion

We explored different ways of formulating and solving Howard’s problem as a reinforcement learning problem. Our first approach was to formulate the problem as an MDP. We solved it using REINFORCE algorithm, and quantified the value of information-seeking actions. The obtained results were close to the the values shown in Howard’s paper. Our second approach was to formulate the problem as a POMDP. We derived and showed that the POMDP can be indeed expressed as an MDP in the belief space. I also tried using a recently proposed POMDP solver package to directly solve Howard’s problem, but unfortunately encountered some runtime errors.

6 Future work

In addition to resolving for the limitations mentioned in the sections describing each approach, further generalizing Howard’s problem accounts for some interesting future work. This can include spy’s accuracy of findings to be proportional to the spy cost, or even more complex paradigms such as more than one company of interest being involved were closed form solution for optimal policy is no longer trivial. The value of information can be evaluated in such complex cases by training an advantage actor critic [4] RL agent that also outputs the value of performing actions. Furthermore, it would be interesting to see if closed form solutions to sub problems can be directly incorporated to aid the RL agent to solve a more complex problem.

References

- [1] R. A. Howard, “Information value theory,” *IEEE Transactions on systems science and cybernetics*, vol. 2, no. 1, pp. 22–26, 1966.
- [2] C. Yoon, “Deriving Policy Gradients and Implementing REINFORCE.” <https://medium.com/@thechrisyoon/deriving-policy-gradients-and-implementing-reinforce-f887949bd63>.
- [3] K. Zheng and S. Tellex, “pomdp-py: A framework to build and solve pomdp problems,” *arXiv preprint arXiv:2004.10099*, 2020.
- [4] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*, pp. 1928–1937, PMLR, 2016.

A Appendices

Code and hyper-parameter values used in sections 3.1, 3.2, and 4.2 is provided here below.

A.1 Code for fixed spy cost, and optimal action on spying

```
import sys
import torch
import gym
import numpy as np
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.autograd import Variable
import matplotlib.pyplot as plt
import pandas as pd
from torch.distributions import Categorical, Normal

# hyperparameters
hidden_size = 25
learning_rate = 1E-1
# Constants
# GAMMA = 0.99
GAMMA = 1
num_steps = 100
# max_episodes = 10000
max_episodes = 10_000

def HowardEnv(state, action, groundTruth):
    flag = action[0]
    bid = action[1]
    lTrue = groundTruth[0]
    pTrue = groundTruth[1]
    costSpy = state
    if flag > .5:
        if lTrue >= bid:
            # reward = (bid - pTrue).detach().numpy()
            reward = (bid - pTrue)
        else:
            reward = 0
        bidFlag = 1
    else:
        if lTrue >= bid:
            bidFlag = 1
            reward = (lTrue - pTrue) - costSpy
```

```

        lFlag = 1
        lEst = lTrue
    else:
        bidFlag = 1
        reward = -costSpy
    return reward

from tqdm import tqdm

# noTimes = 3
# for count in range(noTimes):
num_inputs = 2
num_outputs = 2
batch_size = 5

actor_critic = ActorCritic(num_inputs, num_outputs, hidden_size)
ac_optimizer = optim.Adam(actor_critic.parameters(), lr=learning_rate)

all_rewards = []
ideal_all_rewards = []

bidActions = []
bidAmounts = []

bid_choice_means = []
bid_amt_means = []

for episode in tqdm(range(max_episodes)):
    ps = []
    ls = []
    prob_bid = 0

    actor_loss = 0.
    entropy_loss = 0.
    ac_optimizer.zero_grad()
    for _ in range(batch_size):
        lTrue = 2 * np.random.rand(1)[0]
        pTrue = np.random.rand(1)[0]
        groundTruth = [lTrue, pTrue]

        costSpy = 0.5
#         costSpy = 100
        bidFlag = 0
        lFlag = 0

```

```

lEst = 0
state = costSpy
if episode < 5000:
    distBidStd = .1
else:
    distBidStd = .1

    bid_choice, bid_amt, log_prob, entropy, bid_choice_mean, bid_amt_mean
= actor_critic.forward(state, 0.5, lTrue)
    bidActions.append(bid_choice)
    prob_bid += bid_choice / batch_size
    reward = HowardEnv(state, [bid_choice, bid_amt], groundTruth)
    all_rewards.append(reward)

    ideal_reward = HowardEnv(state, [bid_choice_mean, bid_amt_mean], grou
    ideal_all_rewards.append(ideal_reward)

    reward_t = torch.tensor(reward).float()
    actor_loss = actor_loss - reward_t * log_prob
    entropy_loss = entropy_loss - 0.01 * entropy

    bid_choice_means.append(bid_choice_mean)
#     print(bid_choice_mean) #print to check
    bid_amt_means.append(bid_amt_mean)

    loss = (actor_loss + entropy_loss) / batch_size
    loss.backward()
    ac_optimizer.step()

# Plot results
smoothed_rewards = pd.Series.rolling(pd.Series(all_rewards), 1000).mean()
smoothed_rewards = [elem for elem in smoothed_rewards]
plt.plot(all_rewards)
plt.plot(smoothed_rewards)
plt.plot()
plt.xlabel('Episode')
plt.ylabel('Reward')
plt.show()

```

A.2 Code for varying spy cost, and optimal action on spying

```

import sys
import torch
import gym

```

```

import numpy as np
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.autograd import Variable
import matplotlib.pyplot as plt
import pandas as pd
from torch.distributions import Categorical, Normal

# hyperparameters
hidden_size = 3
learning_rate = 1E-1
# Constants
# GAMMA = 0.99
GAMMA = 1
num_steps = 100
# max_episodes = 10000
max_episodes = 10_000

class ActorCritic(nn.Module):
    def __init__(self, num_inputs, num_actions, hidden_size, learning_rate=3e-4):
        super(ActorCritic, self).__init__()

        self.num_actions = num_actions

        # to spy or not
        self.spyChoice_linear = nn.Linear(num_inputs, num_actions) #shallow
        # self.spyChoice_linear1 = nn.Linear(num_inputs, hidden_size)
        # self.spyChoice_linear2 = nn.Linear(hidden_size, num_actions)

        # self.register_parameter("log_p_bid", nn.Parameter(torch.ones(2)))

        # how much to bid rightaway
        self.register_parameter("mu_bid", nn.Parameter(torch.zeros(1)))

    def forward(self, state, pTrue, lTrue):
        # compute critic
        state_temp = torch.cat((state * torch.ones(1, 1), pTrue * torch.ones(1, 1)))

        # policy_dist_flag = F.softmax(self.spyChoice_linear2(F.relu(self.spyChoice_linear1(state_temp))))
        policy_dist_flag = F.softmax(self.spyChoice_linear(state_temp)) #shallow

        # policy_dist_flag = F.softmax(self.log_p_bid)
        m = Categorical(policy_dist_flag)
        bid_choice = m.sample()

```

```

log_prob = m.log_prob(bid_choice)
entropy = m.entropy()

bid_choice_mean = policy_dist_flag[0][1]
bid_amt_mean = self.mu_bid

if bid_choice == 1:
    # if you choose to bid, use your bidding mean
    bid_dist = Normal(loc=self.mu_bid, scale=0.25 * torch.ones(1))
    bid_amt = bid_dist.sample()
    log_prob = log_prob + bid_dist.log_prob(bid_amt)
else:
    l_temp = lTrue * torch.ones(1, 1)
    # if you choose to spy, use lTrue to get a better bidding mean
    if l_temp > pTrue:
        bid_amt = l_temp
    else:
        bid_amt = l_temp + 100

    return bid_choice, bid_amt, log_prob, entropy, bid_choice_mean, bid_amt_mean

def HowardEnv(state, action, groundTruth):
    flag = action[0]
    bid = action[1]
    lTrue = groundTruth[0]
    pTrue = groundTruth[1]
    costSpy = state
    if flag > .5:
        if lTrue >= bid:
            reward = (bid - pTrue)
        else:
            reward = 0
        bidFlag = 1
    else:
        if lTrue >= bid:
            bidFlag = 1
            reward = (lTrue - pTrue) - costSpy
            lFlag = 1
            lEst = lTrue
        else:
            bidFlag = 1
            reward = -costSpy
    return reward

from tqdm import tqdm

```

```

num_inputs = 2
num_outputs = 2
batch_size = 5

actor_critic = ActorCritic(num_inputs, num_outputs, hidden_size)
ac_optimizer = optim.Adam(actor_critic.parameters(), lr=learning_rate)

all_rewards = []

bidActions = []
bidAmounts = []
SpyCosts = []
bid_choice_means = []
bid_amt_means = []

for episode in tqdm(range(max_episodes)):
    ps = []
    ls = []
    prob_bid = 0

    actor_loss = 0.
    entropy_loss = 0.
    ac_optimizer.zero_grad()
    for _ in range(batch_size):
        lTrue = 2 * np.random.rand(1)[0]
        pTrue = np.random.rand(1)[0]
        groundTruth = [lTrue, pTrue]

#         costSpy = .6 * np.random.rand(1)[0]
        costSpyMax = 1
        costSpy = costSpyMax * np.random.rand(1)[0]
        SpyCosts.append(costSpy)
#         print(costSpy) #print to check
        bidFlag = 0
        lFlag = 0
        lEst = 0
        state = costSpy
        if episode < 5000:
            distBidStd = .1
        else:
            distBidStd = .1
#         distBidStd = max(-.0005*(episode - 5000) + .1, .005)

        bid_choice, bid_amt, log_prob, entropy, bid_choice_mean, bid_amt_mean

```

```

= actor_critic.forward(state, 0.5, lTrue)
    bidActions.append(bid_choice)
    prob_bid += bid_choice / batch_size
    reward = HowardEnv(state, [bid_choice, bid_amt], groundTruth)
    all_rewards.append(reward)
    reward_t = torch.tensor(reward).float()
    actor_loss = actor_loss - reward_t * log_prob
    entropy_loss = entropy_loss - 0.1 * entropy

    bid_choice_means.append(bid_choice_mean)
#     print(bid_choice_mean) #print to check
    bid_amt_means.append(bid_amt_mean)

    loss = (actor_loss + entropy_loss) / batch_size
    loss.backward()
    ac_optimizer.step()

# Plot results
smoothed_rewards = pd.Series.rolling(pd.Series(all_rewards), 1000).mean()
smoothed_rewards = [elem for elem in smoothed_rewards]
plt.plot(all_rewards)
plt.plot(smoothed_rewards)
plt.plot()
plt.xlabel('Episode')
plt.ylabel('Reward')
plt.show()

# Binning
SpyCostsSort = np.sort(SpyCosts)
bid_choice_meansSort = [bid_choice_means[i].detach().numpy() for i in np.argsort(SpyCosts)]
binBid_choice_meansSort = []
noBins = 100
for i in range(noBins):
    binBid_choice_meansSort.append(np.mean(bid_choice_meansSort[int(i * len(SpyCostsSort)) : int((i + 1) * len(SpyCostsSort))]))
plt.plot(np.arange(0, costSpyMax, 1 / noBins).tolist(), binBid_choice_meansSort)
plt.xlabel('Spy Cost')
plt.ylabel('Probability of bidding right away')
plt.show()

```

A.3 Code using POMDP solver package

```

import pomdp-py
from pomdp-py.utils import TreeDebugger
import random

```

```

import numpy as np
import sys

minBidMax = 2
incurredCostMax = 1
costSpyDefault = 10
noBidBins = 3

class CompanyState(pomdp_py.State):
    def __init__(self, minBid, incurredCost):
        self.minBid = minBid
        self.incurredCost = incurredCost
    def __hash__(self):
        return hash((self.minBid, self.incurredCost))
    def __eq__(self, other):
        if isinstance(other, CompanyState):
            return self.minBid == other.minBid and self.incurredCost == other
        return False
    def __str__(self):
        return "%f %f" %(self.minBid, self.incurredCost)
    def __repr__(self):
        return "CompanyState(%f %f)" % (self.minBid, self.incurredCost)

class CompanyAction(pomdp_py.Action):
    def __init__(self, name):
        self.name = name
    def __hash__(self):
        return hash(self.name)
    def __eq__(self, other):
        if isinstance(other, CompanyAction):
            return self.name == other.name
        return False
    def __str__(self):
        return "%s" %(self.name)
    def __repr__(self):
        return "CompanyAction(%s)" %(self.name)

class CompanyObservation(pomdp_py.Observation):
    def __init__(self, minBid):
        self.minBid = minBid
    def __hash__(self):
        return hash(self.minBid)
    def __eq__(self, other):
        if isinstance(other, CompanyObservation):

```

```

        return self.minBid == other.minBid
    return False
def __str__(self):
    return self.minBid
def __repr__(self):
    return "CompanyObservation(%d)" % self.minBid

# Observation model
class ObservationModel(pomdp_py.ObservationModel):

    def sample(self, next_state, action):
        if action.name == "spy":
            return CompanyObservation(next_state.minBid)

# Transition Model
class TransitionModel(pomdp_py.TransitionModel):

    def sample(self, state, action):
        if action.name.startswith("spy"):
            return CompanyState(state.minBid, state.incurringCost)
        else:
            return CompanyState(np.random.uniform(0, minBidMax), n

# Reward Model
class RewardModel(pomdp_py.RewardModel):
    def _reward_func(self, state, action, costSpy):
        if action.name == "spy":
            return -costSpy
        else:
            if float(action.name) > state.minBid:
                return 0
            else:
                return float(action.name) - state.incurringCost
    def sample(self, state, action, next_state):
        # deterministic
        return self._reward_func(state, action, costSpy=costSpyDefault)

# Policy Model
class PolicyModel(pomdp_py.RandomRollout):
    """This is an extremely dumb policy model; To keep consistent
    with the framework."""
    # A stay action can be added to test that POMDP solver is
    # able to differentiate information gathering actions.

```

```

amountACTIONS = {CompanyAction(str(minBidMax * s/noBidBins)) for s in range(1, noBidBins+1)}
ACTIONS = amountACTIONS.add(CompanyAction("spy"))

def sample(self, state, **kwargs):
    if np.random.binomial(1,.5) == 0:
        return "spy"
    else:
        # return random.sample(self.get_all_amountACTIONS(), 1)[0]
        return random.sample(self.get_all_amountACTIONS(), 1)[0]

def get_all_amountACTIONS(self, **kwargs):
    return PolicyModel.amountACTIONS

def get_all_ACTIONS(self, **kwargs):
    return PolicyModel.ACTIONS

class CompanyProblem(pomdp_py.POMDP):
    """
    In fact, creating a TigerProblem class is entirely optional
    to simulate and solve POMDPs. But this is just an example
    of how such a class can be created.
    """

    def __init__(self, init_true_state, init_belief):
        """init_belief is a Distribution."""
        agent = pomdp_py.Agent(init_belief,
                                PolicyModel(),
                                TransitionModel(),
                                ObservationModel(),
                                RewardModel())
        env = pomdp_py.Environment(init_true_state,
                                    TransitionModel(),
                                    RewardModel())
        super().__init__(agent, env, name="CompanyProblem")

    @staticmethod
    def create(state1=minBidMax/2, state2=incurredCostMax/2, belief=1/noBidBins):
        """
        Args:
            state (str): could be 'tiger-left' or 'tiger-right'; True state of the target
            belief (float): Initial belief that the target is on the left; Belief that the target is on the right is 1-belief
            obs_noise (float): Noise for the observation model (default 0.15)
        """
        init_true_state = CompanyState(state1, state2)
        init_belief = pomdp_py.Histogram({CompanyState(minBidMax * (i/(noBidBins+1)),

```

```

    print({CompanyState(minBidMax * (i/(noBidBins-1)),state2): belief for
# print({(i,state2): belief for i in range(0,noBidBins+1)})
    print(init_belief)
    company_problem = CompanyProblem(init_true_state , init_belief)
    company_problem.agent.set_belief(init_belief , prior=True)
    return company_problem

def test_planner(company_problem , planner , nsteps=3, debug_tree=False):
    """
    Runs the action-feedback loop of Tiger problem POMDP

    Args:
        tiger_problem (TigerProblem): an instance of the tiger problem.
        planner (Planner): a planner
        nsteps (int): Maximum number of steps to run this loop.
    """
    for i in range(nsteps):
        action = planner.plan(company_problem.agent)
        if debug_tree:
            from pomdp_py.utils import TreeDebugger
            dd = TreeDebugger(company_problem.agent.tree)
            import pdb; pdb.set_trace()

        print("==== Step %d =====" % (i+1))
        print("True state: %s" % company_problem.env.state)
        print("Belief: %s" % str(company_problem.agent.cur_belief))
        print("Action: %s" % str(action))
        print("Reward: %s" % str(company_problem.env.reward_model.sample(company_problem.env.state)))

        # Let's create some simulated real observation; Update the belief
        # Creating true observation for sanity checking solver behavior.
        # In general, this observation should be sampled from agent's observation model
        real_observation = TigerObservation(company_problem.env.state.name)
        print(">> Observation: %s" % real_observation)
        company_problem.agent.update_history(action , real_observation)

        # If the planner is POMCP, planner.update also updates agent belief.
        planner.update(company_problem.agent , action , real_observation)
        if isinstance(planner , pomdp_py.POUCT):
            print("Num sims: %d" % planner.last_num_sims)
            print("Plan time: %.5f" % planner.last_planning_time)

        if isinstance(company_problem.agent.cur_belief , pomdp_py.Histogram):
            new_belief = pomdp_py.update_histogram_belief(company_problem.agent.cur_belief , real_observation)

```

```

                                                                    action , real_observ
                                                                    company_problem.age
                                                                    company_problem.age

        company_problem.agent.set_belief(new_belief)

    if action.name.startswith("open"):
        # Make it clearer to see what actions are taken until every time
        print("\n")

def main():
    init_true_state = CompanyState(minBidMax/2, incurredCostMax/2)
    init_belief = pomdp_py.Histogram({ CompanyState(minBidMax * (i/(noBidE
    company_problem = CompanyProblem(init_true_state , init_belief)

    print("** Testing value iteration **")
    vi = pomdp_py.ValueIteration(horizon=3, discount_factor=0.95)
    test_planner(company_problem , vi , nsteps=3)

# Reset agent belief
    company_problem.agent.set_belief(init_belief , prior=True)

    print("\n** Testing POUCT **")
    pouct = pomdp_py.POUCT(max_depth=3, discount_factor=0.95,
                            num_sims=4096, exploration_const=50,
                            rollout_policy=company_problem.agent.policy_model ,
                            show_progress=True)
    test_planner(company_problem , pouct , nsteps=10)
    TreeDebugger(company_problem.agent.tree).pp

# Reset agent belief
    company_problem.agent.set_belief(init_belief , prior=True)
    company_problem.agent.tree = None

    print("** Testing POMCP **")
    company_problem.agent.set_belief(pomdp_py.Particles.from_histogram(in
    pomcp = pomdp_py.POMCP(max_depth=3, discount_factor=0.95,
                            num_sims=1000, exploration_const=50,
                            rollout_policy=company_problem.agent.policy_model ,
                            show_progress=True, pbar_update_interval=500)
    test_planner(company_problem , pomcp , nsteps=10)
    TreeDebugger(company_problem.agent.tree).pp

if __name__ == '__main__':
    main()

```